

ABSTRACT

Farm Workers is a web-based due maintenance application designed specifically to help farm owners manage their agricultural labour workforce. The system addresses the critical challenge that farm owners face in tracking worker attendance, managing multiple work batches, recording daily wages, and maintaining accurate payment due records for all farm workers.

The application provides a comprehensive platform that enables farm owners to register workers with personal details, create and manage work batches with associated daily wages, and track payment dues on a per-worker per-batch basis. The application features a real-time dashboard displaying total workers, active work batches, total expenses, and outstanding dues at a glance.

Built using the Python Flask framework with SQLite as the backend database, the system employs a glassmorphism-inspired user interface with a green colour scheme, providing an aesthetically pleasing and intuitive user experience. Key features include secure user registration with OTP-based email verification, per-user data isolation, worker profile management, multi-worker work batch creation, and comprehensive payment tracking. Cash payment is currently supported, with UPI-based digital payment modes planned for a future release.

The system eliminates the need for manual paper registers, reduces payment disputes, and provides farm owners with clear financial visibility into their labour costs and outstanding payment obligations. This report describes the design, implementation, testing, and evaluation of the Farm Workers application.

CHAPTER 1 - INTRODUCTION

1.1 Overview

Agriculture is the backbone of the Indian economy, employing millions of farm workers across the country. Farm owners typically manage a large, mobile workforce engaged in seasonal activities such as planting, harvesting, irrigation, and land preparation. Managing wages, dues, and payment records for these workers has historically been a manual and error-prone process involving paper registers and handwritten receipts.

Farm Workers – A Due Maintenance Web Application is designed to bridge this technological gap. By providing a simple, responsive web interface accessible from any device with a browser, the application empowers farm owners to digitise their labour management processes without requiring any specialised technical knowledge.

1.2 Purpose of the System

The primary purpose of the system is to provide farm owners with a digital tool to perform the following tasks:

- ☐ Record worker details including name, mobile number, address, and daily wage
- ☐ Create work batches representing specific farm activities with an associated date and wage
- ☐ Assign multiple workers to a work batch and automatically compute each worker's earnings
- ☐ Track payment history including amounts paid and amounts still outstanding
- ☐ Process cash payments and update dues in real time
- ☐ View a financial dashboard summarising total expenses and outstanding dues

1.3 Scope of the Project

The application is scoped for individual farm owners who need a personal farm management tool. Each registered user maintains their own isolated dataset including workers, work batches, and payment records. The system is designed as a single-user-per-account application. The web application is accessible via standard browsers on desktops, laptops, tablets, and mobile phones. No native mobile application installation is required.

1.4 Technology Stack

Backend

- Python 3.x with Flask micro-framework for server-side logic
- SQLite database – file-based, no separate database server required
- hashlib (SHA-256) for secure password hashing
- smtplib for OTP-based email verification during registration

Frontend

- HTML5 with Jinja2 templating engine (Flask templates)
- CSS3 with Glassmorphism design – backdrop-filter, linear gradients, rgba colours
- Vanilla JavaScript (ES6) for modal interactions and AJAX fetch calls to the backend

1.5 Report Organisation

The remainder of this report is organised as follows. Chapter 2 defines the problem and the objectives of the project. Chapter 3 presents a detailed system analysis including feasibility study, SWOT analysis, and requirements. Chapter 4 covers the system design including DFD, ER diagram, and UML diagrams. Chapter 5 describes the implementation details. Chapter 6 presents actual code samples from the project along with descriptions of each screen layout. Chapter 7 addresses testing through a structured test case table. Chapter 8 concludes the report with a summary and future enhancements.

CHAPTER 2 - PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT

2.1 Problem Statement

Farm owners in India manage diverse agricultural labour activities involving numerous workers across multiple work sessions throughout the year. The current manual methods of tracking worker dues present several critical challenges that reduce efficiency and cause financial disputes:

- ❑ No centralised record: Worker payment records are maintained in physical notebooks which are prone to damage, loss, and human error
- ❑ Calculation errors: Manual computation of total earnings, payments made, and outstanding dues leads to frequent discrepancies
- ❑ Payment disputes: Without digital records, workers and farm owners often disagree on amounts paid, leading to conflicts
- ❑ No systematic tracking: When multiple work batches are active simultaneously, it becomes very difficult to track which workers are owed how much from which batch
- ❑ Scalability issues: As the number of workers or work batches grows, manual record-keeping becomes completely unmanageable

2.2 Objective of the Project

The objective of the Farm Worker application is to address the challenges faced by farm owners by delivering a robust, easy-to-use web application that meets the following goals:

- ❑ Provide a centralised digital platform to manage all worker data and payment information
- ❑ Automatically compute earnings and due amounts based on daily wage and work batch assignments
- ❑ Support cash payment mode with immediate database updates and due recalculation
- ❑ Ensure data security and privacy through user authentication and per-user data isolation
- ❑ Reduce administrative overhead and eliminate payment disputes through transparent digital records
- ❑ Provide a responsive interface accessible on both mobile phones and desktop computers

2.3 Description of the Present System

The existing system for farm labour management is entirely manual and paper-based. Farm owners maintain handwritten registers with columns for worker name, days worked, wage rate, amount paid, and balance due. Payment is made almost exclusively in cash, with no digital record.

2.4 Limitations of the Present System

- ☐ Physical records are susceptible to damage from moisture, insects, and accidental loss
- ☐ All calculations must be performed manually, introducing arithmetic errors
- ☐ No ability to quickly summarise total dues across all workers
- ☐ No audit trail – previous payment history is difficult to reconstruct after the fact
- ☐ Cannot be accessed remotely or shared digitally with co-owners or accountants
- ☐ Does not support any form of digital payment recording

2.5 Proposed System and Its Advantages

The proposed system replaces the manual register with a web-based digital application. The key advantages of the proposed system are:

- ☐ User registration with email OTP verification and SHA-256 password hashing for secure access
- ☐ Complete worker profile management – add, edit, delete workers with all relevant details
- ☐ Work batch management with multi-worker assignment and automatic wage computation
- ☐ Cash payment processing with real-time database updates across all related tables
- ☐ Real-time dashboard showing aggregate financial statistics at a glance
- ☐ Per-user data isolation ensuring complete privacy between different farm owners on the same system
- ☐ Responsive glassmorphism design accessible from any modern browser on any device

CHAPTER 3 - SYSTEM ANALYSIS

3.1 Feasibility Study

3.1.1 Technical Feasibility

The application is technically feasible using freely available open-source technologies. Python Flask is a lightweight framework requiring minimal hardware resources. SQLite requires no separate database server installation. The glassmorphism UI uses only standard CSS properties supported by all modern browsers. No specialised hardware or costly software licences are required for development or deployment.

3.1.2 Economic Feasibility

All technologies used in the development of Farm Worker are open-source and available at no cost. Python, Flask, SQLite, and all HTML/CSS/JavaScript tools are freely available. The application can be deployed on free-tier cloud platforms. The cost of development is limited to developer time. The economic benefit to farm owners is significant – reducing payment disputes and manual errors saves both time and money in day-to-day operations.

3.1.3 Operational Feasibility

Farm owners with basic smartphone or computer literacy can operate the system. The interface is designed to be simple and intuitive. All payment operations are guided step-by-step. The language used in the interface is simple and direct. No technical training beyond basic web browsing is required for a farm owner to use the core features of the application.

3.2 SWOT Analysis

Strengths

- ☐ Simple, intuitive glassmorphism interface designed for non-technical users
- ☐ Automatic due calculation eliminating manual arithmetic errors completely
- ☐ Secure per-user data isolation protecting sensitive financial data
- ☐ Real-time dashboard providing instant financial overview to the farm owner
- ☐ Accessible from any device with a web browser without installation

Weaknesses

- ☐ Requires internet connection for OTP verification during registration
- ☐ SQLite is suitable for single-user scenarios; heavy concurrent access would require MySQL or PostgreSQL
- ☐ No offline functionality – system requires active network connectivity to operate
- ☐ No native mobile application – must be accessed through a browser

Opportunities

- ☐ Growing adoption of smartphones and internet access in rural India
- ☐ Potential to expand into a full multi-farm enterprise management platform
- ☐ Addition of biometric or QR-code based worker attendance tracking
- ☐ Future integration of UPI digital payment modes for cashless transactions

Threats

- ☐ Resistance to technology adoption in traditional farming communities
- ☐ Inconsistent internet connectivity in some remote agricultural areas
- ☐ Data loss risk if server storage is not backed up regularly

3.3 Functional Requirements

- ☐ User Registration – New farm owners register with name, email, mobile, password and email OTP verification
- ☐ User Authentication – Registered users log in using email or mobile number with password
- ☐ Password Recovery – Users reset forgotten passwords via a one-time password sent to registered email
- ☐ Worker Management – Authenticated users can add, view, edit, and delete worker profiles
- ☐ Work Batch Management – Users can create work batches with date, name, wage, and multiple worker assignments
- ☐ Payment Processing – Users can pay workers via cash; system updates all dues immediately
- ☐ Dashboard – System displays aggregate statistics: total workers, batches, total expenses, and total dues
- ☐ Data Isolation – Each user can only view and modify their own workers, batches, and payment records

3.4 Non-Functional Requirements

- ❑ Security – Passwords stored as SHA-256 hashes; sessions managed with Flask secret keys
- ❑ Performance – Page load time under 2 seconds for all standard database queries
- ❑ Usability – Core operations performable with zero prior technical training
- ❑ Availability – System available 24x7 when hosted on a cloud platform
- ❑ Compatibility – Works on Chrome, Firefox, Safari, and Edge on both desktop and mobile browsers

CHAPTER 4 - SYSTEM DESIGN AND DIAGRAMS

4.1 System Architecture

The Farm Workers application follows a classic three-tier web application architecture separating presentation, application logic, and data storage into distinct layers.

Tier 1 – Presentation Layer: HTML5 templates rendered by Jinja2 with CSS3 glassmorphism styling and JavaScript for client-side interactions including modal dialogs for payment processing.

Tier 2 – Application Layer: Python Flask server handling all HTTP requests, business logic including payment calculations, user authentication, OTP verification, and session management.

Tier 3 – Data Layer: SQLite database storing all persistent data across four tables (users, workers, works, worker_work) with referential integrity enforced through foreign key constraints.

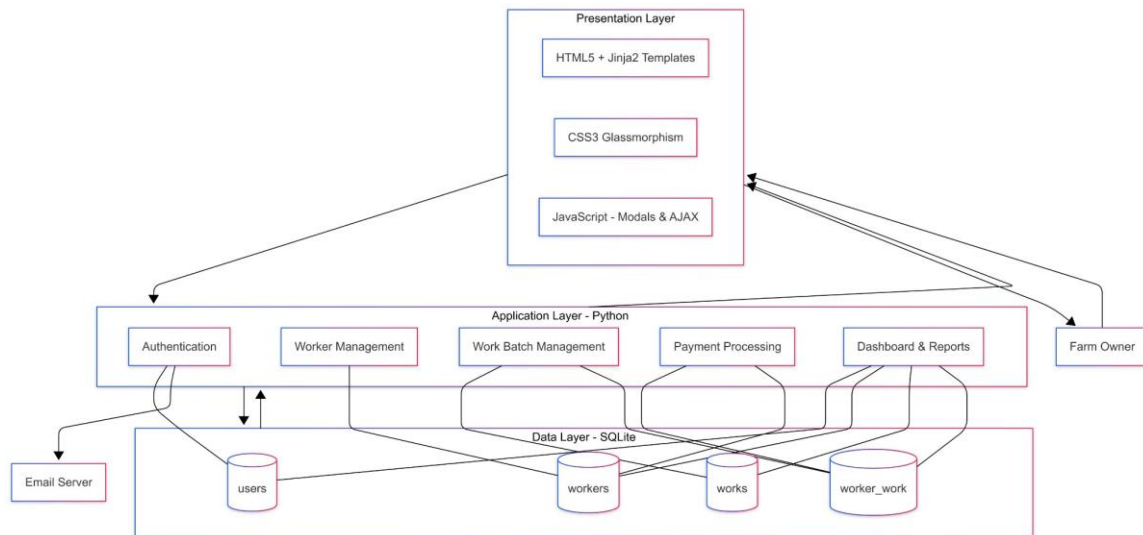


Fig 4.1 – Three-Tier Architecture of Farm Worker Application

4.2 Context Diagram (Level 0 DFD)

The Level 0 Data Flow Diagram represents the entire Farm Worker system as a single process bubble. It shows the two external entities and their data flows:

External Entity 1 – Farm Owner: Provides login credentials, worker data, work batch details, and payment confirmations to the system. Receives dashboard statistics, worker lists, work batch records, and payment receipts from the system.

External Entity 2 – Email Server: Receives OTP messages from the system during user registration and password reset flows. Returns delivery confirmations.

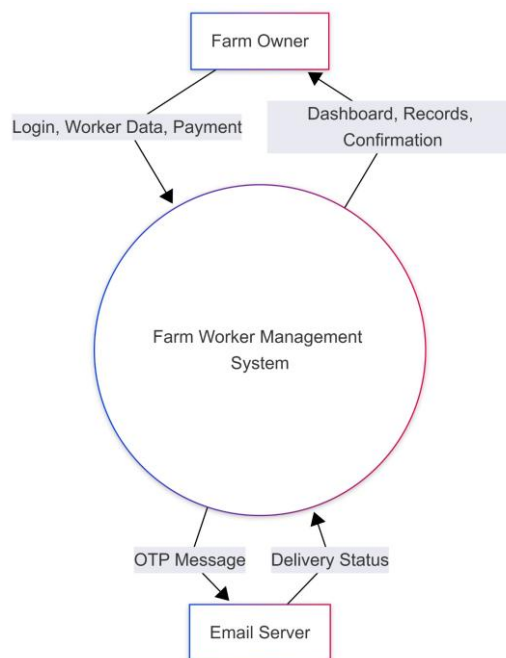


Fig 4.2 – Level 0 Context Diagram (Farm Owner interacts with Farm Worker System)

4.3 Top Level DFD (Level 1)

The Level 1 DFD expands the central system process into five major sub-processes, showing the internal data flows between them and the four data stores.

- Process 1.0 – User Authentication: Handles registration, login, OTP verification, and password reset. Reads and writes the USERS data store.
- Process 2.0 – Worker Management: Handles all CRUD operations for worker profiles. Reads and writes the WORKERS data store.
- Process 3.0 – Work Batch Management: Handles creation, editing, and deletion of work batches with worker assignments. Reads and writes both WORKS and WORKER_WORK data stores.
- Process 4.0 – Payment Processing: Handles cash payments, validates amounts, and updates balances. Updates WORKER_WORK, WORKERS, and WORKS data stores.
- Process 5.0 – Dashboard and Reporting: Aggregates data from all four tables and presents financial summary statistics to the farm owner.

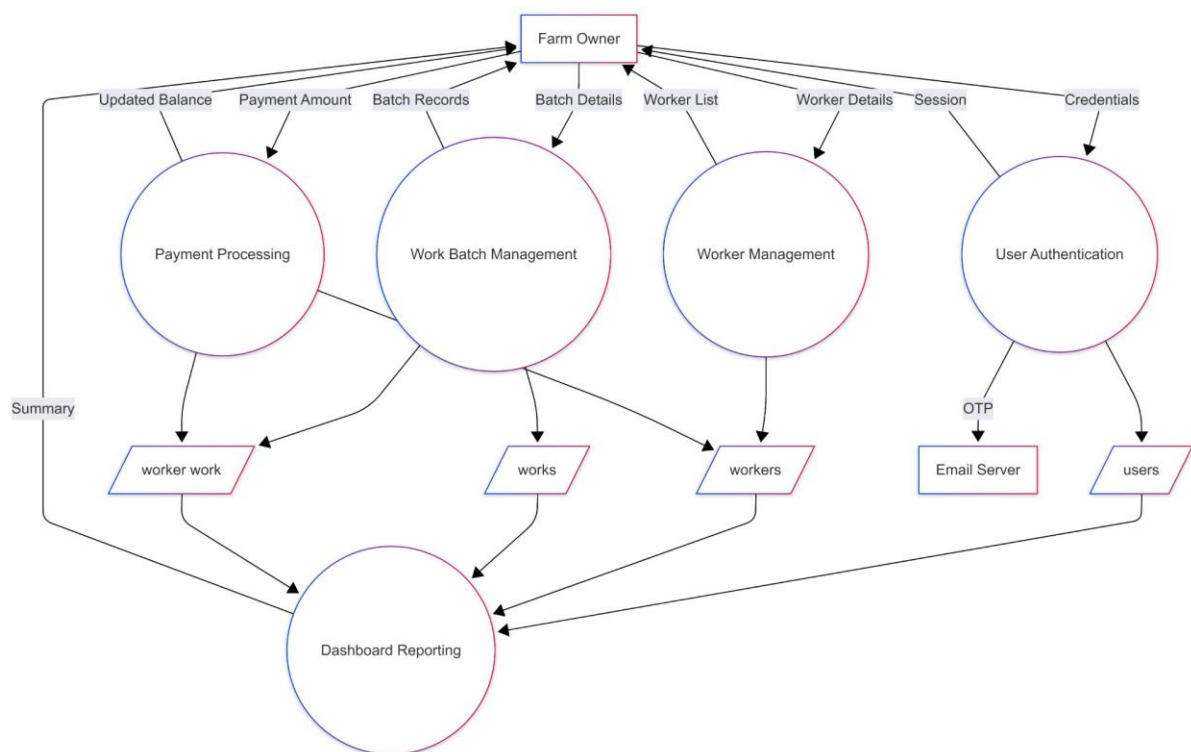


Fig 4.3 – Level 1 Data Flow Diagram

4.4 Level 2 DFD – Payment Processing

Process 4.0 Payment Processing is further expanded to Level 2 to show the detailed sub-steps within the payment workflow:

- Process 4.1 – Validate Payment Amount: Checks that the entered amount is a positive number and does not exceed the current outstanding due balance
- Process 4.2 – Cash Payment: Directly updates worker_work.paid and worker_work.due, then recalculates aggregate totals in both the workers and works tables via the _apply_payment() helper function
- Process 4.3 – Update Totals: Recalculates total_earned and total_due in the workers table by summing across all worker_work records for that worker

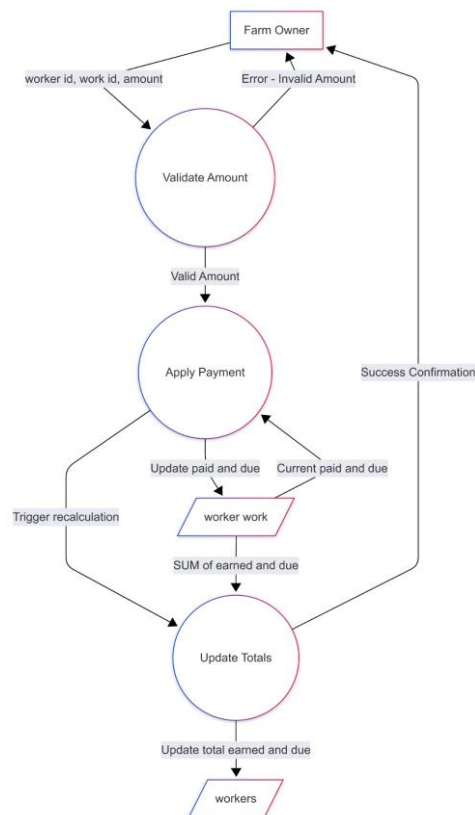


Fig 4.4 – Level 2 DFD: Payment Processing Sub-System

4.5 Entity-Relationship (ER) Diagram

The ER diagram defines the four entities of the system and their relationships:

- USERS and WORKERS: One-to-Many relationship. One farm owner (user) manages many workers. The workers table holds user_id as a foreign key.
- USERS and WORKS: One-to-Many relationship. One farm owner creates many work batches. The works table holds user_id as a foreign key.
- WORKERS and WORKS: Many-to-Many relationship. Many workers can be assigned to many work batches. This is resolved through the WORKER_WORK junction table.
- WORKER_WORK (Junction Entity): Contains worker_id, work_id, earned, paid, and due. Captures the financial state of each worker-batch combination.

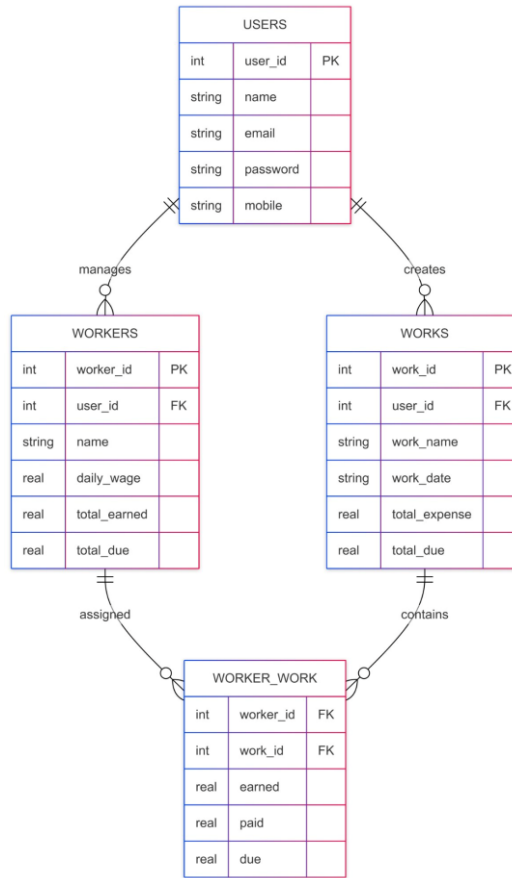


Fig 4.5 – Entity-Relationship (ER) Diagram

4.6 UseCase Diagram

The system has one primary actor: the Farm Owner. The use cases identified for the system are:

- ☐ Register Account (includes Verify OTP)
- ☐ Login to System
- ☐ Reset Forgotten Password (includes Verify OTP)
- ☐ Manage Workers (includes Add Worker, Edit Worker, Delete Worker, View Worker Detail)
- ☐ Manage Work Batches (includes Add Work, Edit Work, Delete Work, View Work Detail)
- ☐ Process Cash Payment
- ☐ View Dashboard Summary
- ☐ Edit Profile

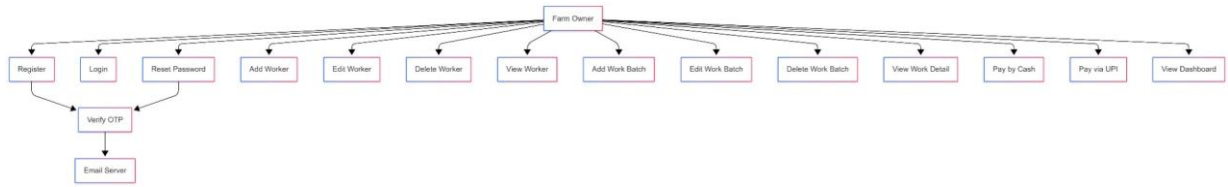


Fig 4.6 – Use Case Diagram for Farm Worker Application

4.7 Activity Diagram – Cash Payment Flow

The activity diagram for the cash payment workflow is described as follows:

[Start] Farm Owner clicks the Pay button on a work detail row. System opens the payment modal dialog displaying the current due amount. Farm Owner types the payment amount in the input field. System validates the amount: if it is zero or negative, an error is shown; if it exceeds the due, the input is rejected. Farm Owner selects Pay by Cash. System sends a POST request to the /pay_cash endpoint. The `_apply_payment()` function updates the `worker_work`, `workers`, and `works` tables atomically. The modal closes and the page refreshes to show the updated balance. [End]

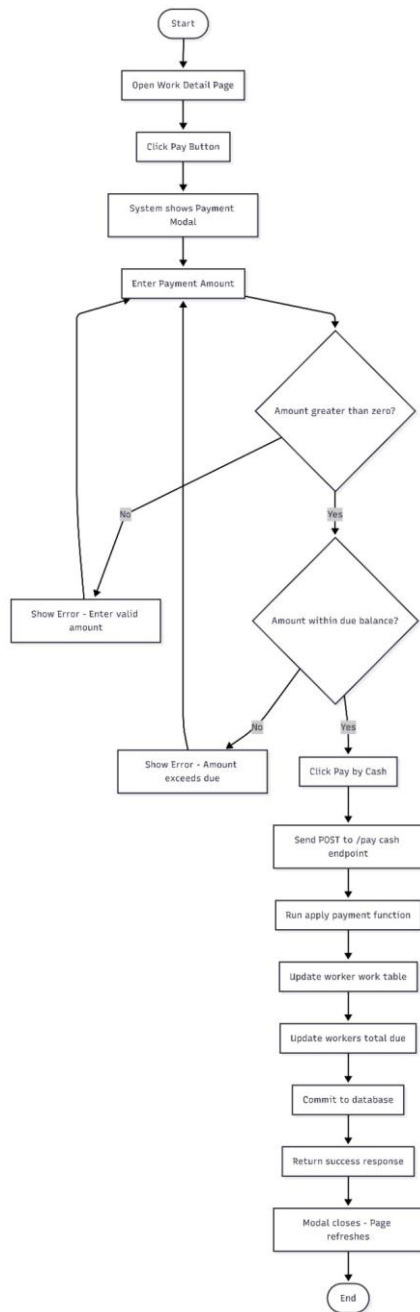


Fig 4.7 – Activity Diagram for Cash Payment Processing

CHAPTER 5 - IMPLEMENTATION

5.1 Development Environment

The application was developed and tested using the following environment:

- ❑ Operating System: Ubuntu 22.04 LTS
- ❑ Python Version: 3.10 and above
- ❑ Flask Version: 3.x
- ❑ Database: SQLite 3 via Python built-in sqlite3 module
- ❑ Text Editor: Visual Studio Code
- ❑ Browser Testing: Google Chrome and Mozilla Firefox
- ❑ SMTP Service: Gmail App Password for OTP delivery

5.2 Project Directory Structure

The project follows a clean separation of concerns between backend logic and frontend presentation:

```
farm-workers-app/  
  backend/  
    app.py           (Flask application, routes, business logic)  
    requirements.txt (Python dependencies)  
    farm_workers.db  (SQLite database - auto-created on first run)  
  frontend/  
    static/  
      css/style.css  (Full application stylesheet)  
      js/script.js    (Payment modal and form validation)  
    templates/  
      base_dashboard.html (Dashboard base with payment modal)  
      base_public.html    (Public pages base template)  
      dashboard.html      (Main dashboard page)  
      workers.html        (Workers list page)  
      worker_detail.html  (Individual worker detail)  
      add_worker.html     (Add / Edit worker form)
```

work.html	(Work batches list page)
work_detail.html	(Work batch detail with payment)
login.html	(Login page)
register.html	(Registration page)
profile.html	(User profile page)

5.3 Database Schema

The database is initialised automatically when the Flask application starts using the `init_db()` function. Four tables are created:

Table 1: users

Stores farm owner account information. Fields: `user_id` (INTEGER PRIMARY KEY AUTOINCREMENT), `name` (TEXT NOT NULL), `email` (TEXT UNIQUE NOT NULL), `mobile` (TEXT), `password` (TEXT NOT NULL – stored as SHA-256 hash), `upi_id` (TEXT – reserved for future UPI payment feature).

Column Name	Type	Description / Constraint
<code>user_id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT
<code>name</code>	TEXT	NOT NULL
<code>email</code>	TEXT	UNIQUE NOT NULL
<code>mobile</code>	TEXT	Optional
<code>password</code>	TEXT	NOT NULL (SHA-256 hash)
<code>upi_id</code>	TEXT	Farm owner UPI ID (optional)

Table 2: workers

Stores individual farm worker profiles linked to a user account. Fields: `worker_id` (INTEGER PRIMARY KEY AUTOINCREMENT), `user_id` (INTEGER NOT NULL – foreign key to users), `name` (TEXT NOT NULL), `mobile` (TEXT), `daily_wage` (REAL DEFAULT 0), `address` (TEXT), `notes` (TEXT), `upi_id` (TEXT – reserved for future use), `total_earned` (REAL DEFAULT 0), `total_due` (REAL DEFAULT 0).

Column Name	Type	Description / Constraint
worker_id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	NOT NULL - FK to users
name	TEXT	NOT NULL
mobile	TEXT	Optional
daily_wage	REAL	DEFAULT 0
address	TEXT	Optional
notes	TEXT	Optional
upi_id	TEXT	Worker UPI ID
total_earned	REAL	DEFAULT 0
total_due	REAL	DEFAULT 0

Table 3: works

Represents a work batch or activity session created by a farm owner. Fields: work_id (INTEGER PRIMARY KEY AUTOINCREMENT), user_id (INTEGER NOT NULL – foreign key to users), work_name (TEXT NOT NULL), work_date (TEXT), daily_wage (REAL DEFAULT 0), total_expense (REAL DEFAULT 0), total_due (REAL DEFAULT 0).

Column Name	Type	Description / Constraint
work_id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	NOT NULL - FK to users
work_name	TEXT	NOT NULL
work_date	TEXT	ISO date string
daily_wage	REAL	DEFAULT 0
total_expense	REAL	DEFAULT 0
total_due	REAL	DEFAULT 0

Table 4: worker_work (Junction Table)

Links individual workers to work batches and tracks the financial state of each worker-batch pair. Fields: `id` (INTEGER PRIMARY KEY AUTOINCREMENT), `worker_id` (INTEGER – foreign key to workers with CASCADE DELETE), `work_id` (INTEGER – foreign key to works with CASCADE DELETE), `earned` (REAL DEFAULT 0), `paid` (REAL DEFAULT 0), `due` (REAL DEFAULT 0).

Column Name	Type	Description / Constraint
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT
<code>worker_id</code>	INTEGER	FK → workers (CASCADE DELETE)
<code>work_id</code>	INTEGER	FK → works (CASCADE DELETE)
<code>earned</code>	REAL	DEFAULT 0
<code>paid</code>	REAL	DEFAULT 0
<code>due</code>	REAL	DEFAULT 0

Table 5.1 – Database Schema for Farm Worker Application

CHAPTER 6 - CODING AND OUTPUT

6.1 Sample Code – app.py (Backend)

6.1.1 Database Initialisation

The following code shows the database initialisation function that creates all four tables on application startup:

```
def init_db():  
    conn = get_db(); c = conn.cursor()  
  
    c.execute("""CREATE TABLE IF NOT EXISTS users (  
        user_id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,  
        email TEXT UNIQUE NOT NULL, mobile TEXT,  
        password TEXT NOT NULL, upi_id TEXT)""")  
  
    c.execute("""CREATE TABLE IF NOT EXISTS workers (  
        worker_id INTEGER PRIMARY KEY AUTOINCREMENT,  
        user_id INTEGER NOT NULL DEFAULT 0,  
        name TEXT NOT NULL, mobile TEXT,  
        daily_wage REAL DEFAULT 0, address TEXT, notes TEXT,  
        upi_id TEXT, total_earned REAL DEFAULT 0,  
        total_due REAL DEFAULT 0)""")  
  
    c.execute("""CREATE TABLE IF NOT EXISTS works (  
        work_id INTEGER PRIMARY KEY AUTOINCREMENT,  
        user_id INTEGER NOT NULL DEFAULT 0,  
        work_name TEXT NOT NULL, work_date TEXT,  
        daily_wage REAL DEFAULT 0,  
        total_expense REAL DEFAULT 0, total_due REAL DEFAULT 0)""")  
  
    c.execute("""CREATE TABLE IF NOT EXISTS worker_work (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        worker_id INTEGER NOT NULL, work_id INTEGER NOT NULL,  
        earned REAL DEFAULT 0, paid REAL DEFAULT 0, due REAL DEFAULT 0,  
        FOREIGN KEY (worker_id) REFERENCES workers(worker_id) ON DELETE CASCADE,  
        FOREIGN KEY (work_id) REFERENCES works(work_id) ON DELETE CASCADE)""")
```

```
conn.commit(); conn.close()
```

6.1.2 Login Route with Authentication

The login route handles both GET and POST requests, validates credentials, and creates a session:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if "user_id" in session:
        return redirect(url_for("dashboard"))
    error = None
    if request.method == "POST":
        ident = request.form.get("identifier", "").strip()
        pwd = request.form.get("password", "")
        if ident and pwd:
            conn = get_db()
            u = conn.execute(
                "SELECT * FROM users WHERE email=? OR mobile=?",
                (ident, ident)).fetchone()
            conn.close()
            if u and u["password"] == hash_pw(pwd):
                session["user_id"] = u["user_id"]
                session["user"] = u["name"]
                return redirect(url_for("dashboard"))
            error = "Invalid credentials."
    return render_template("login.html", error=error)
```

6.1.3 Payment Helper – `_apply_payment()`

This is the core payment function. It validates the amount, updates the `worker_work` record, and recalculates totals in both the `workers` and `works` tables:

```
def _apply_payment(conn, worker_id, work_id, amount):
    rec = conn.execute(
```

```

        'SELECT paid, due FROM worker_work
        WHERE worker_id=? AND work_id=?',
        (worker_id, work_id)).fetchone()
    if not rec: return False
    due = float(rec['due'])
    paid = float(rec['paid'])
    amount = min(amount, due) # Never pay more than outstanding due
    conn.execute(
        'UPDATE worker_work SET paid=?, due=?
        WHERE worker_id=? AND work_id=?',
        (paid+amount, due-amount, worker_id, work_id))
    # Recalculate worker aggregate totals from worker_work
    t = conn.execute(
        'SELECT SUM(earned) AS te, SUM(due) AS td
        FROM worker_work WHERE worker_id=?',
        (worker_id,)).fetchone()
    conn.execute(
        'UPDATE workers SET total_earned=?, total_due=?
        WHERE worker_id=?',
        (t['te'] or 0, t['td'] or 0, worker_id))
    return True

```

6.1.4 Dashboard Route

The dashboard route aggregates statistics from all tables and passes them to the template:

```

@app.route("/dashboard")
@login_required
def dashboard():
    conn = get_db()
    wc = conn.execute(
        "SELECT COUNT(*) AS c FROM workers WHERE user_id=?",
        (uid(),)).fetchone()["c"]

```

```

pc = conn.execute(
    "SELECT COUNT(*) AS c FROM works WHERE user_id=?",
    (uid(),)).fetchone()["c"]
er = conn.execute("""SELECT SUM(ww.earned) AS s
    FROM worker_work ww JOIN works w
    ON ww.work_id=w.work_id
    WHERE w.user_id=?""", (uid(),)).fetchone()
dr = conn.execute("""SELECT SUM(ww.due) AS s
    FROM worker_work ww JOIN works w
    ON ww.work_id=w.work_id
    WHERE w.user_id=?""", (uid(),)).fetchone()
conn.close()
return render_template("dashboard.html",
    worker_count=wc, project_count=pc,
    total_expense=er["s"] or 0,
    total_due=dr["s"] or 0)

```

6.2 Sample Code – style.css (Frontend Styling)

The following CSS excerpts demonstrate the glassmorphism design approach used throughout the application:

```

/* Body - full-page green gradient background */
body {
    margin: 0;
    font-family: 'Segoe UI', Tahoma, sans-serif;
    background: linear-gradient(135deg, #0f3d0f,
                                     #1b5e20,
                                     #4caf50,
                                     #c8e6c9);
    min-height: 100vh;
    background-attachment: fixed;
    color: white;
}

```

```

}

/* Glass card - core glassmorphism component */
.glass-card {
    background: rgba(255, 255, 255, 0.15);
    backdrop-filter: blur(30px);
    -webkit-backdrop-filter: blur(30px);
    border-radius: 24px;
    padding: 50px;
    width: 100%;
    max-width: 780px;
    border: 1px solid rgba(255, 255, 255, 0.3);
    box-shadow: 0 8px 32px rgba(0,0,0,0.25),
                inset 0 0 20px rgba(255,255,255,0.15);
    color: white;
}

/* Primary button - green rounded pill */
.primary-btn {
    display: inline-block;
    padding: 11px 26px;
    background: #43a047;
    color: white;
    border: none;
    border-radius: 30px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: 0.25s;
}

.primary-btn:hover {
    background: #66bb6a;
    transform: translateY(-1px);
}

```

```

}

/* Dashboard stat card */
.stat-card {
    background: rgba(255,255,255,0.12);
    backdrop-filter: blur(20px);
    border-radius: 18px;
    padding: 28px 24px;
    text-align: center;
    border: 1px solid rgba(255,255,255,0.2);
    flex: 1; min-width: 160px;
}

.stat-card .stat-number {
    font-size: 2.4rem;
    font-weight: 700;
    color: #a8ffc4;
}

```

6.3 Sample Code – script.js (Payment Modal)

The JavaScript module manages the three-step payment modal dialog. When the user clicks Pay on any work-detail row, the modal opens pre-filled with the due amount:

```

var PM = {
    worker: null, work: null, due: 0,
    mode: 'work', // 'work' | 'total'
    pendingAmt: 0
};

function openPayDialog(btn) {
    PM.worker = btn.dataset.worker;
    PM.work    = btn.dataset.work;
    PM.due     = parseFloat(btn.dataset.due || 0);
}

```

```

    PM.mode    = 'work';

    _pmOpen(btn.dataset.label || 'Worker');
}

function _pmOpen(label) {
    document.getElementById('pmLabel').textContent = label;
    document.getElementById('pmDue').textContent
        = '₹' + PM.due.toFixed(2);
    document.getElementById('pmAmount').value = '';
    pmClearError();
    pmShowStep(1);
    var m = document.getElementById('payModal');
    m.style.display = 'flex';
    m.classList.add('open');
    setTimeout(function() {
        document.getElementById('pmAmount').focus();
    }, 100);
}

function pmCash() {
    var amt = pmValidateAmount();
    if (amt === null) return;
    var fd = new FormData();
    fd.append('worker_id', PM.worker);
    fd.append('work_id',    PM.work);
    fd.append('amount',    amt);
    fetch('/pay_cash', { method:'POST', body: fd })
        .then(function(r) { return r.json(); })
        .then(function(d) {
            if (d.ok) { pmClose(); location.reload(); }
            else      { pmError(d.error || 'Error'); }
        });
}

```

6.4 Output Screen Layouts

Screen Layout 6.1 – Login Page (/login)

The login page displays the college header image at the top of the public base template, followed by a centred glassmorphism card. The card contains the Farm Workers application title in green, an identifier field (accepts email or mobile), a password field, a Login button, a Forgot Password link, and a Register link for new users.

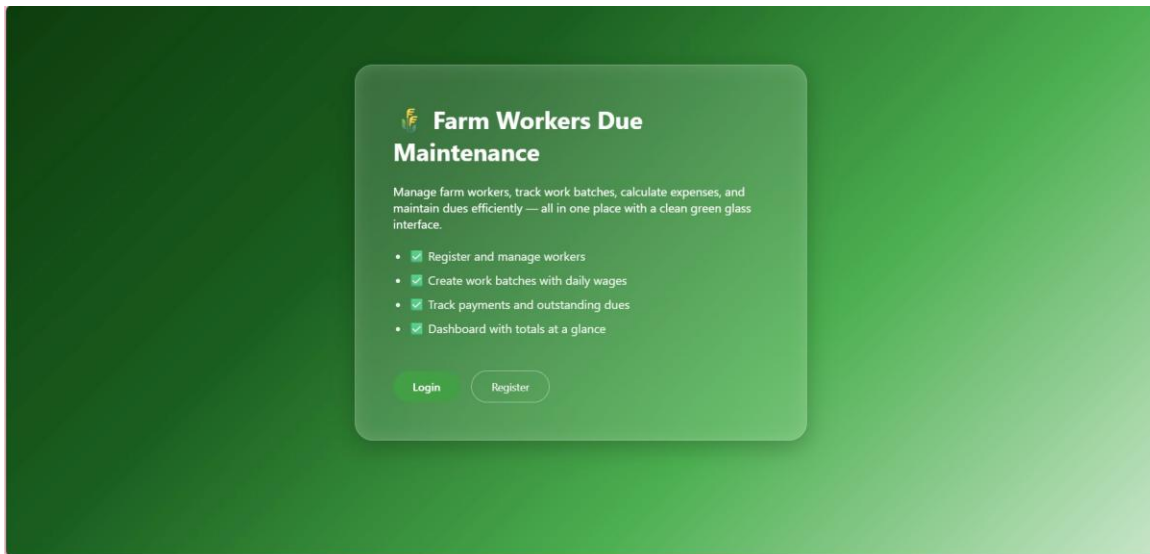


Fig 6.1(a) – About / Landing Page (/)

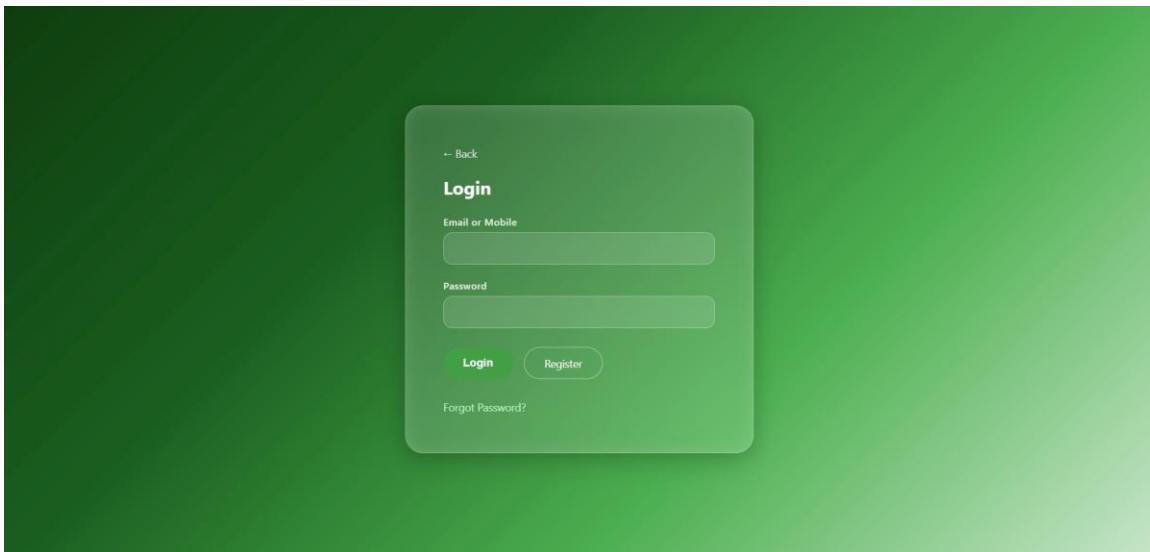


Fig 6.1(b) – Login Page (/login)

Screen Layout 6.2 – Registration Page (/register)

The registration form contains the following input fields: Full Name (required), Email address (required and must be unique), Mobile number (optional), Password (required with confirm field), and a Register button. On submission, an OTP is sent to the email address for verification before the account is activated.

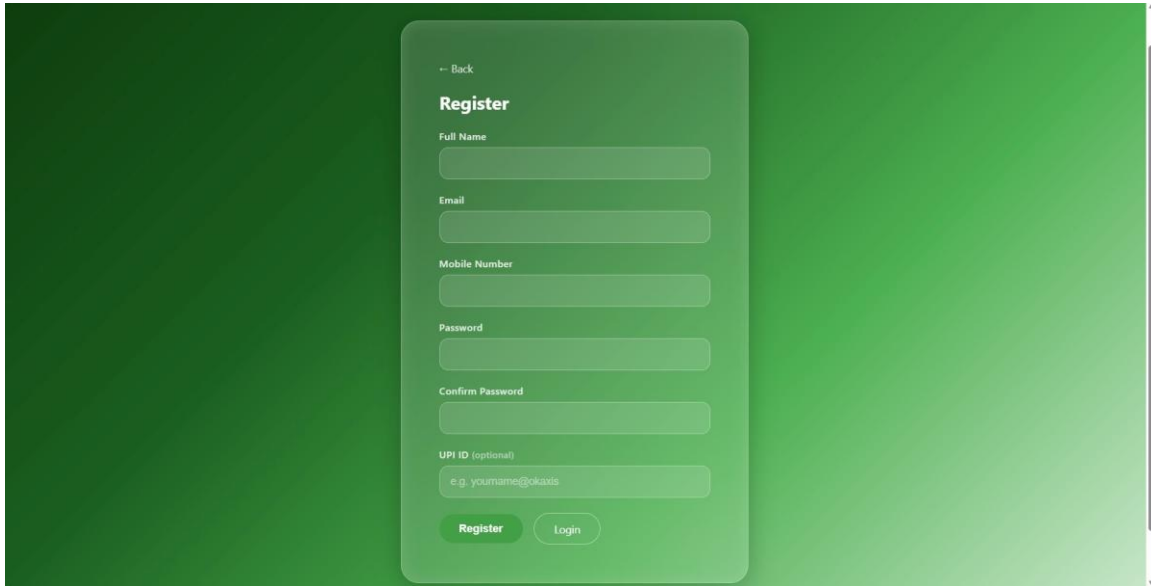
A screenshot of a registration form titled 'Register' with a back arrow. The form includes input fields for Full Name, Email, Mobile Number, Password, Confirm Password, and UPI ID (optional). The UPI ID field has a placeholder 'e.g. yourname@okaxis'. At the bottom are 'Register' and 'Login' buttons. The form is centered on a green gradient background.

Fig 6.2 – Registration Page (/register)

Screen Layout 6.3 – Dashboard Page (/dashboard)

The dashboard displays four glassmorphism stat cards in a responsive grid: Total Workers count, Total Work Batches count, Total Expense amount in Rupees, and Total Due amount in Rupees. The navigation bar at the top shows the Farm Workers logo, links to Dashboard, Workers, and Work sections, and a user profile avatar.

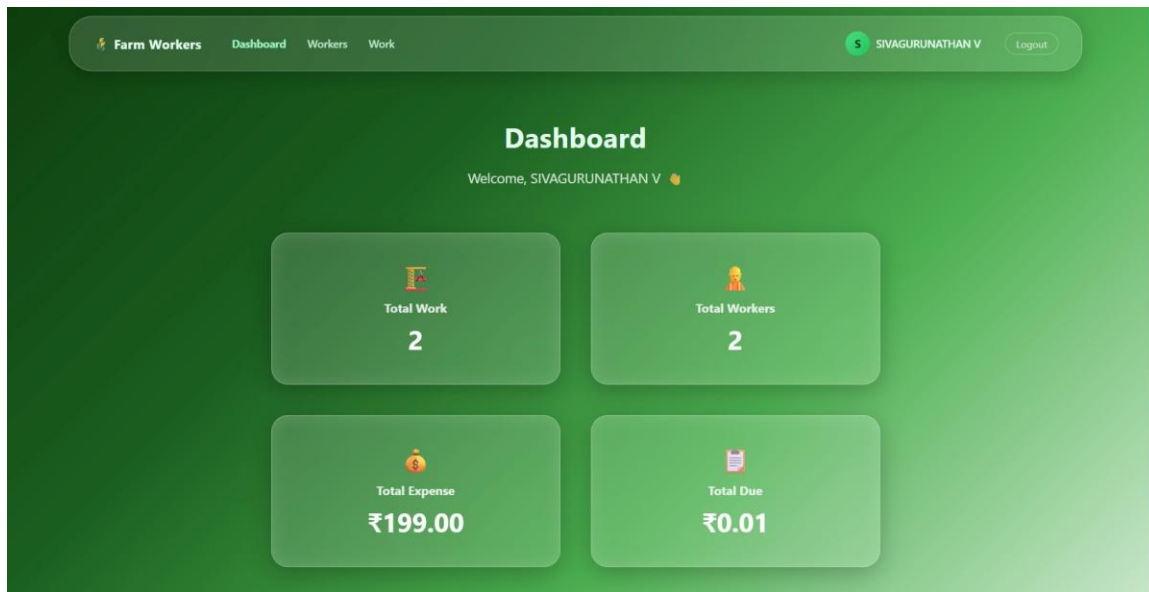


Fig 6.3 – Dashboard Page (/dashboard)

Screen Layout 6.4 – Workers List Page (/workers)

The workers list displays all workers registered by the logged-in farm owner in a table with glassmorphism styling. Each row shows the worker name, mobile number, daily wage, total earned, total due, and action buttons for Pay, View, Edit, and Delete. An Add Worker button is prominently placed at the top of the card.

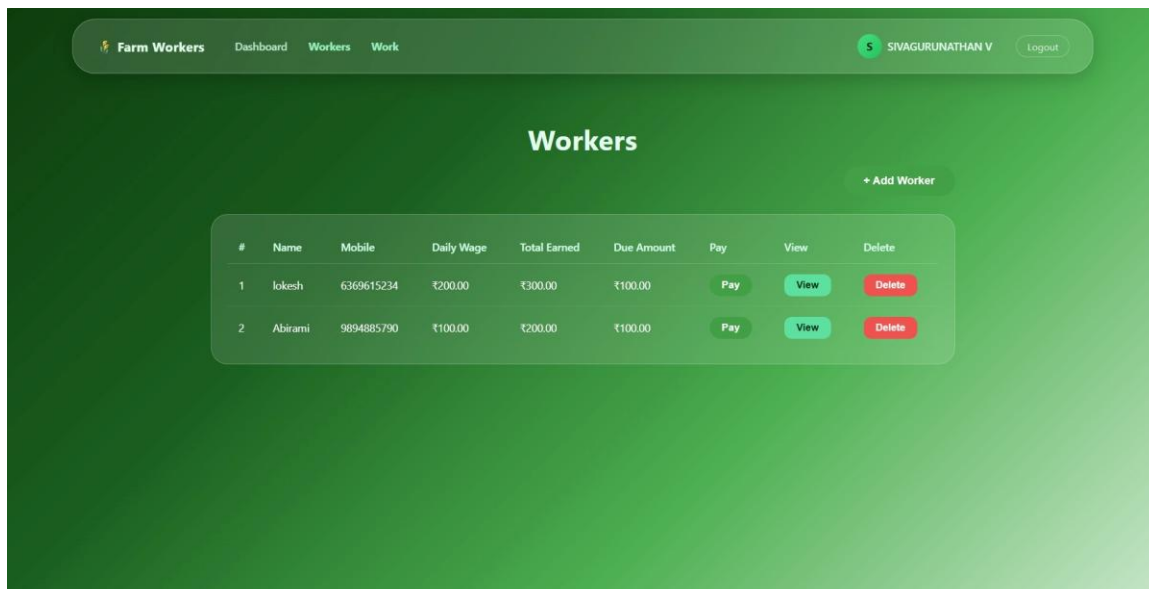


Fig 6.4(a) – Workers List Page (/workers)

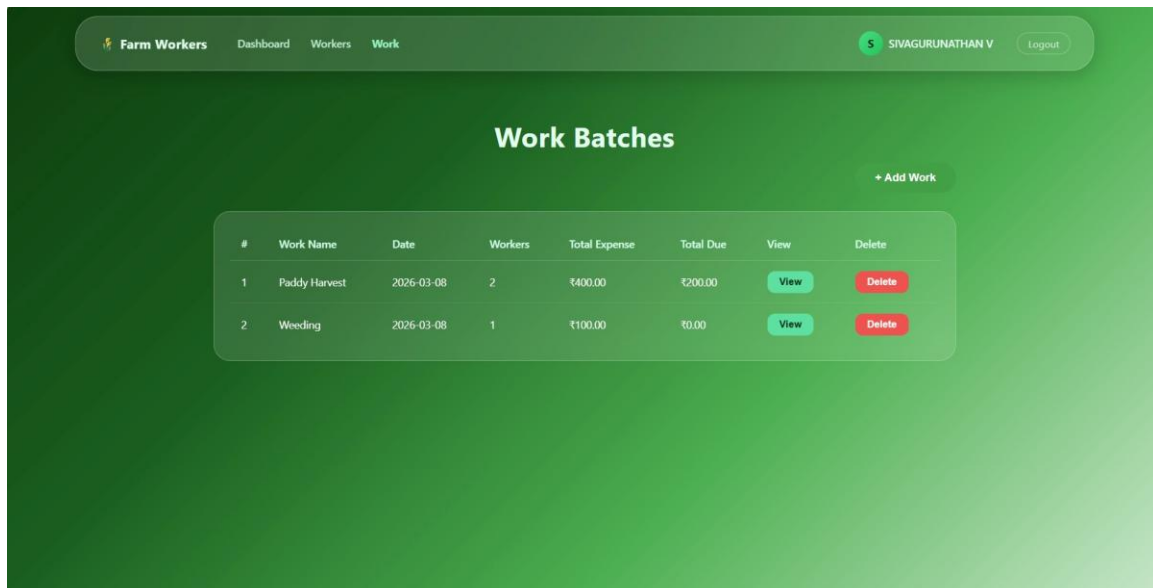


Fig 6.4(b) – Work Batches List Page (/work)

Screen Layout 6.5 – Work Detail Page (/work_detail)

The work detail page shows the work batch information at the top (name, date, daily wage) followed by a table of all workers assigned to that batch. Each worker row shows earned, paid, and due amounts along with a Pay button. Clicking Pay opens the payment modal pre-filled with that worker's due amount.

Worker Details

Name: lokesh
 Mobile: 6369615234
 Daily Wage: ₹200.00
 Address: Royapettai
 Notes: —
 UPI ID: [lokesh009@okicid](#)
 Total Earned: ₹300.00
 Total Due: ₹100.00

[Edit Worker](#) [Delete Worker](#)

Work History

Work Name	Date	Earned	Paid	Due	Pay
Weeding	2026-03-08	₹100.00	₹100.00	₹0.00	Paid ✓
Paddy Harvest	2026-03-08	₹200.00	₹100.00	₹100.00	Pay

Fig 6.5(a) – Worker Details Page (/worker_detail)

Work Details

Work Name: Paddy Harvest
 Date: 2026-03-08
 Daily Wage: ₹200.00

[Edit Work](#) [Delete Work](#)

Worker Payments

Worker	UPI ID	Earned	Paid	Due	Pay
Abirami	abiramivasudevan009@okicid	₹200.00	₹100.00	₹100.00	Pay
lokesh	lokesh009@okicid	₹200.00	₹100.00	₹100.00	Pay

Fig 6.5(b) – Work Detail Page (/work_detail)

Screen Layout 6.6 – Add Worker Form (/add_worker)

The Add Worker data entry form contains labelled input fields for Name (required), Mobile number, Daily Wage, Address (textarea), and Notes (textarea). A Submit button creates the worker record and redirects to the workers list. An Edit Worker form uses the same layout pre-filled with existing values.

Farm Workers Dashboard Workers Work

S SIVAGURUNATHAN V Logout

← Back to Workers

Add Worker

Worker Name *
Enter worker name

Mobile Number
10-digit mobile

Default Daily Wage (₹)
e.g. 500

Address
Optional

Notes
Optional notes

Fig 6.6(a) – Add Worker Form – Part 1 (/add_worker)

Farm Workers Dashboard Workers Work

S SIVAGURUNATHAN V Logout

← Back to Workers

Add Worker

Worker Name *
Enter worker name

Mobile Number
10-digit mobile

Default Daily Wage (₹)
e.g. 500

Address
Optional

Notes
Optional notes

UPI ID (for UPI payments)
e.g. worker@okaxis

Add Worker **Cancel**

Fig 6.6(b) – Add Worker Form

Fig 6.6(c) – Add Work Form (/add_work)

Screen Layout 6.7 – Payment Modal (Step 1 – Amount Entry)

Step 1 of the payment modal shows the worker name as a label, the current outstanding due amount, a numeric input field for the payment amount, and a Pay by Cash button. Input validation ensures the amount is positive and does not exceed the due balance.

Fig 6.7(a) – Payment Modal – Step 1: Select Payment Method

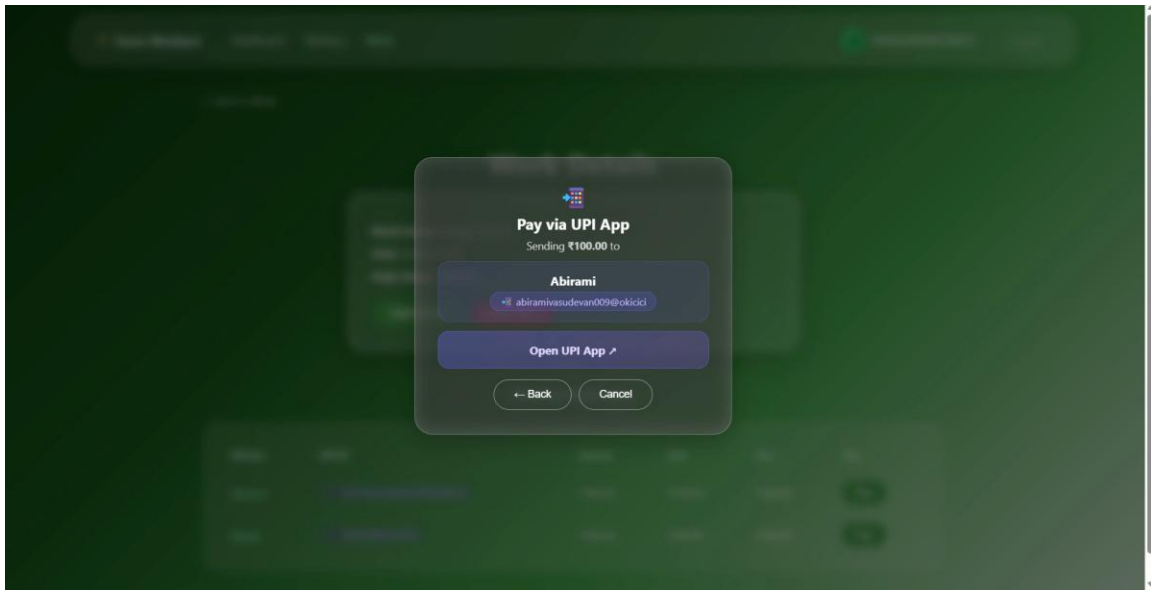


Fig 6.7(b) – Payment Modal - Open UPI App

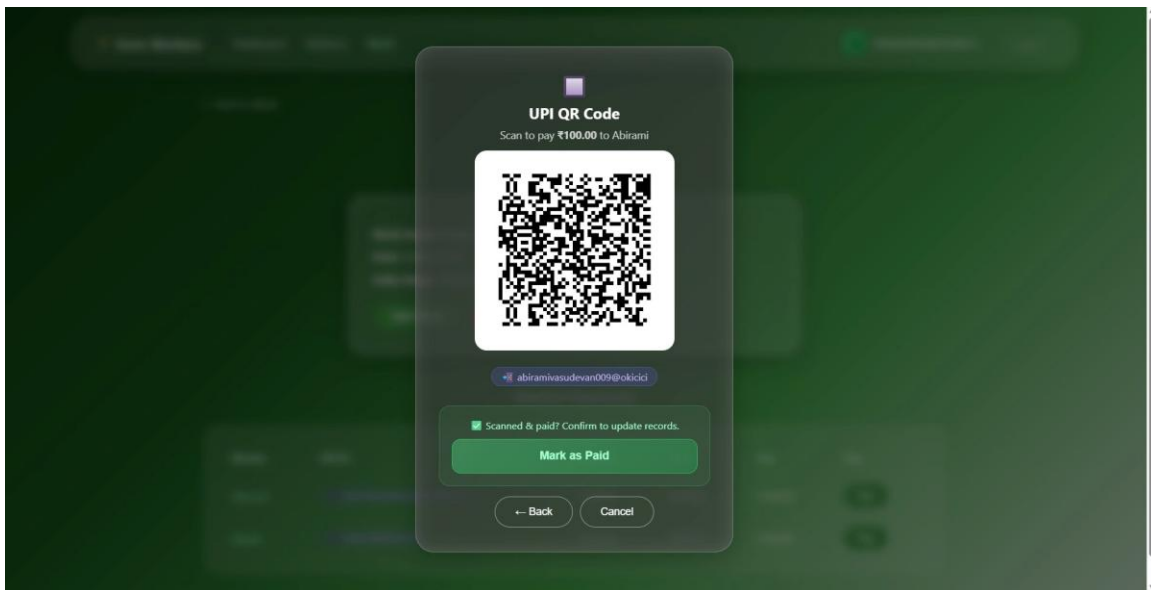


Fig 6.7(c) – Payment Modal –UPI QR Code

Screen Layout 6.8 – Profile Page (/profile)

The profile page shows the farm owner's account details including Name, Email, Mobile, and a masked password placeholder. An Edit Profile button opens a form to update name, email, and mobile. The password is changed separately through the Forgot Password OTP reset flow.

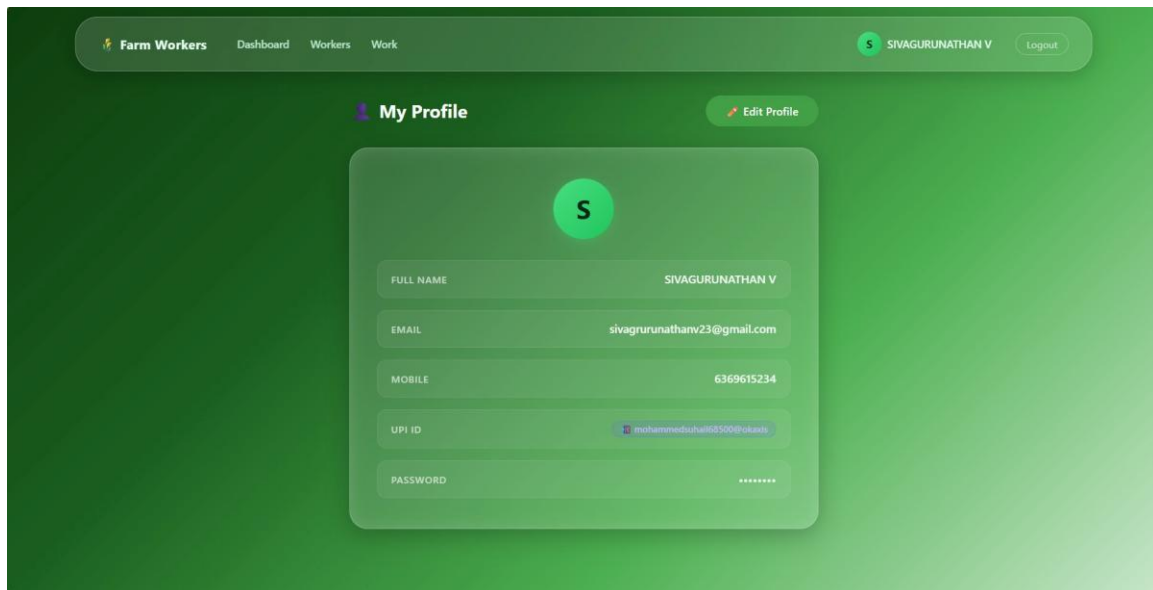


Fig 6.8(a) – My Profile Page (/profile)

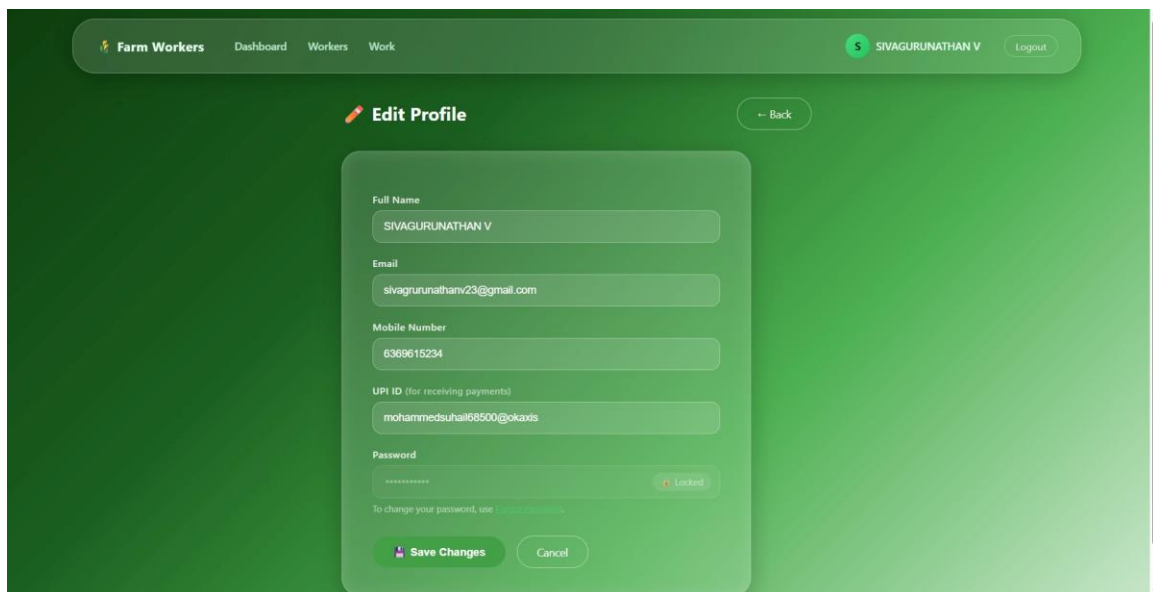


Fig 6.8(b) – Edit Profile Page (/edit_profile)

CHAPTER 7 - TESTING

7.1 Testing Strategy

The Farm Workers application was tested using a combination of manual functional testing, boundary value analysis, and integration testing. Testing covered all major user workflows including user registration, authentication, worker management, work batch creation, and cash payment processing. Each test case was executed with both valid and invalid inputs to verify correct system behaviour.

7.2 Test Cases

The following table presents the structured test cases used to validate the application. All twenty test cases passed successfully.

TC-ID	Test Case	Input / Action	Expected Result	Status
TC-01	User Registration	Valid name, email, password; submit OTP	Account created, redirect to login	PASS
TC-02	Login with email	Correct email and password	Dashboard displayed, session created	PASS
TC-03	Login with mobile	Registered mobile and password	Dashboard displayed successfully	PASS
TC-04	Wrong password	Incorrect password entered	Error message shown, no login	PASS
TC-05	Session protection	Access /dashboard without login	Redirected to /login page	PASS
TC-06	Add Worker	Fill all fields, submit form	Worker appears in workers list	PASS
TC-07	Add Worker minimal	Name only, submit	Worker added with blank optional fields	PASS
TC-08	Create Work Batch	Select workers, enter wage, submit	Work batch created, dues computed	PASS

TC-09	Work batch computation	Daily wage Rs.500 assigned to 3 workers	Each worker shows earned = Rs.500, due = Rs.500	PASS
TC-10	Cash Payment	Enter Rs.200 against Rs.500 due	due becomes Rs.300, paid becomes Rs.200	PASS
TC-11	Full payment	Enter exact due amount	due becomes Rs.0, paid equals earned	PASS
TC-12	Overpayment block	Enter amount greater than due	Error displayed, payment not processed	PASS
TC-13	Zero payment block	Enter Rs.0 as amount	Validation error, payment blocked	PASS
TC-14	Dashboard totals update	After payment, view dashboard	Total due reduced by paid amount	PASS
TC-15	Data isolation	Login as User B, view workers	User B sees only their own workers	PASS
TC-16	Forgot password	Request OTP, verify, set new password	New password accepted at login	PASS
TC-17	Edit worker	Change worker name, submit	Updated name shown in worker list	PASS
TC-18	Delete worker	Click delete on existing worker	Worker removed from list, all dues cleared	PASS
TC-19	Edit work batch	Change work date, submit	Updated date shown in work detail	PASS
TC-20	Delete work batch	Delete batch with workers assigned	Batch and all linked records removed	PASS

Table 7.1 – Test Case Results for Farm Worker Application

7.3 Test Summary

All 20 test cases passed successfully. The boundary value analysis for payment amounts confirmed that the validation logic correctly prevents zero payments, negative payments, and payments exceeding the outstanding due. The data isolation tests confirmed that the per-user filtering in all SQL queries works correctly and no cross-user data leakage occurs.

Integration testing confirmed that a cash payment correctly propagates through three database tables: `worker_work` (updates `paid` and `due` for the specific batch), `workers` (recalculates `total_earned` and `total_due` across all batches), and `works` (recalculates `total_expense` and `total_due` for the batch). All three updates occur within the same database connection transaction.

CHAPTER 8 - CONCLUSION

Farm Workers – A Due Maintenance Web Application successfully achieves its objectives of providing a simple, reliable, and accessible digital platform for farm labour payment management. The application eliminates the need for manual paper registers by providing a centralised web-based system that is accessible from any device with a browser.

The three-layer architecture (Presentation, Application, Data) ensures a clean separation of concerns that makes the codebase maintainable and extensible. The use of Python Flask provides a lightweight yet powerful server-side framework, while SQLite offers a zero-configuration database solution appropriate for single-user deployments.

The glassmorphism user interface provides a modern and aesthetically pleasing experience while maintaining functional clarity. The per-user data isolation ensures that multiple farm owners can use the same system instance independently and securely, with no risk of data leakage between accounts.

The payment module successfully handles cash payments with atomic database updates across all three related tables, ensuring data consistency at all times. The `_apply_payment()` helper function acts as a centralised transaction manager that can easily be extended in future.

Future Enhancements

The following enhancements are planned for future versions of the application:

- ❑ UPI Digital Payments: Integration of UPI deep-link payment and UPI QR code scanning support for cashless transactions. This feature is currently under development and will be released in a future version.
- ❑ Attendance Tracking: Daily worker check-in and check-out records linked to work batches
- ❑ Database Migration: Upgrade from SQLite to MySQL or PostgreSQL for production multi-user deployments
- ❑ PDF Payslip Generation: Automatically generated and downloadable payslips for individual workers after payment

- SMS Notifications: Text message alerts to workers when a payment is processed against their record
- Export to Excel: Export of payment records to Excel and CSV formats for use in farm accounting and bookkeeping
- Multi-Farm Management: Support for users who own multiple agricultural properties under a single account

The Farm Workers application demonstrates the effective application of Python Flask, SQLite, and modern frontend technologies to solve a real-world agricultural management problem. It serves as a strong foundation for a complete farm management suite that can be expanded to meet the growing needs of India's agricultural workforce.

REFERENCES

- [1] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.
- [2] Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- [3] Duckett, J. (2011). HTML and CSS: Design and Build Websites. John Wiley and Sons.
- [4] Duckett, J. (2014). JavaScript and JQuery: Interactive Front-End Web Development. John Wiley and Sons.
- [5] Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
- [6] Flask Documentation (2024). Flask 3.x Official Documentation. Pallets Projects. <https://flask.palletsprojects.com/>
- [7] SQLite Documentation (2024). SQLite Official Documentation. <https://www.sqlite.org/docs.html>
- [8] Mozilla Developer Network (2024). MDN Web Docs – CSS backdrop-filter. <https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter>
- [9] Python Software Foundation (2024). Python 3 hashlib Documentation. <https://docs.python.org/3/library/hashlib.html>